Interrupciones externas en el ESP32

Una **interrupción externa** avisa a la CPU cuando cambia un pin. La **ISR** debe ser **corta**, ir en **IRAM**, usar variables `volatile` y el **patrón bandera**.

Hardware · Software · código · Regla · decisión · Error · ojo · ◇ decisión

1 Del botón al loop()

mientras corre la ISR, loop() espera

1 — Se presiona el botón BOTÓN

Con `INPUT_PULLUP` el pin pasa de 1 a 0: flanco **FALLING**.

```
pin 21: 1 → 0
```

2 — El GPIO lo detecta GPIO + CPU

El hardware vigila el pin, aunque `loop()` esté en un `delay()`. La CPU salta a la ISR.

3 — Corre la ISR ISR

Antirrebote ≥ 50 ms, sube la bandera y sale.

```
void IRAM_ATTR isrBoton()
```

4 — Bandera `volatile` VARIABLE

`alerta = true` significa «pasó algo».

```
volatile bool alerta
```

5 — `loop()` trabaja LOOP()

Primero baja la bandera y después hace el trabajo pesado. Llega tarde, **pero no se pierde**.

2 FALLING, RISING o CHANGE

cuándo debe actuar la ISR

↓ **Con `INPUT_PULLUP`: suelto = 1, presionado = 0**

Al presionar
La señal baja de 1 a 0.
`FALLING`

Al soltar
La señal sube de 0 a 1.
`RISING`

En ambos
La ISR corre **dos veces por toque**.
`CHANGE`

CASO TÍPICO

Botón con `INPUT_PULLUP` y alarma al presionar → **FALLING**.

3 Cuándo lleva volatile

depende de cuántas partes usan la variable

1. **La usa la ISR** — No → **No necesita `volatile`**
2. Sí → **La usa también `loop()`** — No → **No necesita `volatile`**
3. Sí → **Lleva `volatile`**

Obliga a leer la variable de la RAM cada vez. Sin él, `loop()` nunca ve el cambio y **no hay error de compilación**.

```
volatile bool alerta = false;
```

Lleva volatile

`EJEMPLO` `alerta`, `pulsos`: la ISR escribe y el loop lee

vs

No lleva

`EJEMPLO` `tUltimo` (solo la ISR), `total` (solo el loop)

4 IRAM_ATTR

toda ISR lo lleva

ISR sin `IRAM_ATTR`	Queda en la flash.
Caché apagada + llega una interrupción	La **caché** es la memoria rápida que lee la flash. La CPU no puede leer la ISR.
El chip se cae	Pasa a veces, no siempre.

SIEMPRE

`void IRAM_ATTR isrBoton() { … }` copia la ISR a la **IRAM**, una RAM interna.

QUÉ VA EN LA ISR

✔ SÍ

- Subir una bandera
- Guardar `millis()`
- Sumar 1 a un contador
- `digitalWrite`

✘ NO

- `Serial.print` / `println`
- `delay()`
- Cálculos largos

POR QUÉ

Una ISR larga hace que el **watchdog** (vigila que el programa no se trabe) reinicie el chip.

PINES CON INTERRUPCIÓN (ESP32; EN EL UNO SOLO D2 Y D3)

Sirven	**16** **17** **18** **19** **21** **22** **23** **25** **26** **27** **32** **33** Los recomendados. El botón del código va en GPIO21.
Con cuidado	**34** **35** **36** **39** Solo entrada y **sin pull-up interna**: usar **10 kΩ a 3,3 V**.
Evitar	**0** **2** **5** **12** **15** Pines de strapping: si están presionados al encender, arranca mal.
No	**6–11** flash Los usa la memoria del programa.

CÓDIGO: ALERTA CON BOTÓN

```
const uint8_t PIN_BOTON = 21;
const uint8_t PIN_LED   = 19;
const uint32_t T_REBOTE = 50;           // ms
volatile bool alerta = false;           // ISR y loop
uint32_t tUltimo = 0;                   // solo la ISR

void IRAM_ATTR isrBoton() {
  uint32_t ahora = millis();
  if (ahora - tUltimo >= T_REBOTE) {
    alerta = true;
    tUltimo = ahora;
  }
}
void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_BOTON, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_BOTON), isrBoton, FALLING);
}
void loop() {
  if (alerta) {
    alerta = false;
    digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    Serial.println("¡Alerta!");
  }
  delay(2000);                          // loop ocupado: el aviso NO se pierde
}
```

ATTACHINTERRUPT

- Una sola vez, en `setup()`
- **Después** de `pinMode()`
- La ISR va **sin paréntesis**

SIN ANTIRREBOTE

10 toques cuentan **más de 10**: cada toque rebota y da varios flancos de bajada.